Git Guide.

THE COMPANION

Every Git and GitHub answer in one place.
The model, the commands, and the rescues, printed to keep beside you.

an undo for everything · works offline · no trackers · MIT

Amey Thakur
amey-thakur.github.io/GIT-GUIDE

A note from Amey

Git carries nearly every codebase on earth, and almost everyone who uses it learned it by accident: a command from a teammate, a midnight search, a ritual repeated without understanding.

Git Guide exists to end that way of learning. Ask in plain language, or paste the error Git printed, and get the exact commands with the two things most resources never give: how dangerous each one is, and how to take it back.

This companion is the still version. The model on the next pages replaces memorizing; the command pages cover the daily work; the rescue pages are for the bad days. The living version, with every answer, every error, and a place to ask, waits at **amey-thakur.github.io/GIT-GUIDE**.

Amey

Where your code lives

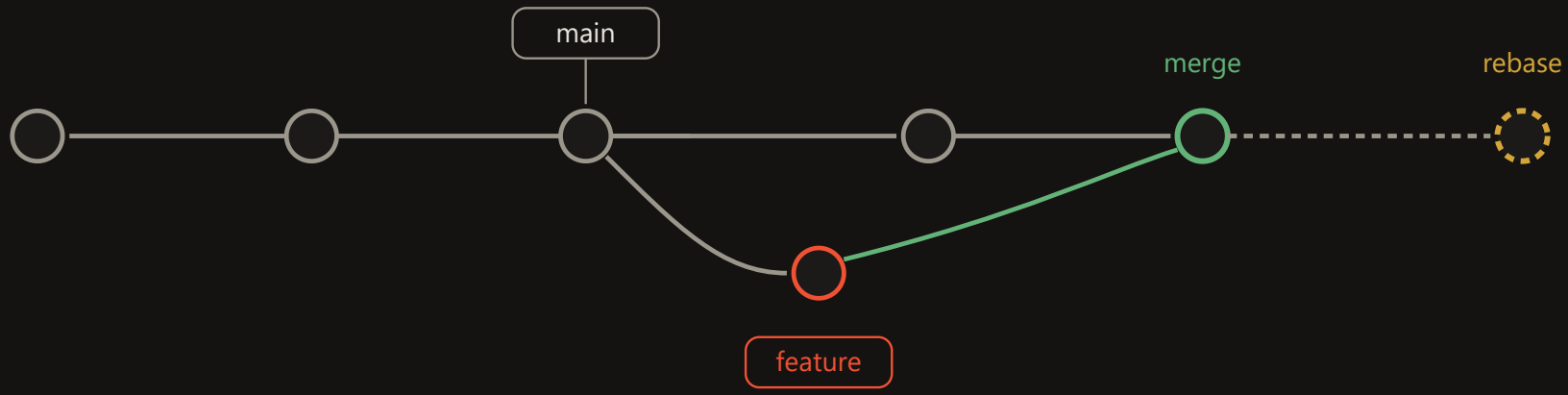
add copies a change into the staging area, the draft of your next commit.

commit seals the draft into a permanent snapshot in your local repository.

push and **fetch** move commits to and from the remote; **pull** is fetch plus merge.

restore copies recorded versions back over your files; **reset** moves the branch pointer itself.

Commits, branches, HEAD



A **branch** is a movable label on one commit. Creating one copies nothing.

A **rebase** replays your commits as new ones for a straight line. Rewrites history: unshared work only.

A **merge** commit has two parents; history shows what truly happened. Safe on shared branches.

HEAD is where you stand. Checking out a commit directly detaches it; that is a state, not an error.

The daily loop

`git status -sb`	What changed, one line per file
`git diff --staged`	What the next commit will contain
`git commit -m "<message>"`	Seal the staged snapshot
`git fetch`	See what the remote has before touching anything
`git push`	Publish your commits
`git diff`	Unstaged edits, line by line
`git add <file>`	Stage a change; git add -p picks piece by piece
`git commit --amend --no-edit`	Add staged changes to the last commit
`git pull --rebase`	Get remote commits, replay yours on top

Branches

`git switch -c <branch>`	Create and move in one step
`git switch -`	Back to the branch you just left
`git branch -vv`	Every local branch with its remote and ahead-behind state
`git push -u origin <branch>`	Publish a new branch
`git push origin --delete <branch>`	Delete it on the remote too
`git switch <branch>`	Move to an existing branch
`git branch --show-current`	Where am I
`git branch -a`	Every branch, local and remote
`git branch -d <branch>`	Delete a merged branch; -D forces
`git branch -m <old> <new>`	Rename

Undo and rescue

`git restore <file>`	Discard edits, back to last commit
`git commit --amend -m "<new message>"`	Rewrite the last commit message
`git reset --hard HEAD~1`	Erase the last commit and its work
`git reflog`	Every state you have been in; the safety net
`git clean -nd`	Preview deleting untracked files; -fd deletes
`git restore --staged <file>`	Unstage, keep the edits
`git reset --soft HEAD~1`	Uncommit, keep everything staged
`git revert <hash>`	Undo by adding an opposite commit; safe when pushed
`git reset --hard ORIG_HEAD`	Undo the merge, rebase, or pull that just happened

Stash

`git stash push -u -m "<label>"`	Set everything aside, untracked included
`git stash pop`	Bring the newest back and drop it
`git stash show -p`	See inside before applying
`git stash list`	What is shelved
`git stash apply stash@{<n>}`	Bring a specific one back, keep it shelved

Merge and rebase

`git merge <branch>`	Bring a branch in; history stays true
`git rebase main`	Replay your branch on top of main
`git cherry-pick <hash>`	Copy one commit onto this branch
`git rebase --continue`	Resume after resolving a conflict
`git merge --no-ff <branch>`	Always keep the feature visible as one bubble
`git rebase -i HEAD~<n>`	Rewrite, reorder, squash your last n commits
`git merge --abort`	Back out of a conflicted merge

Inspect and search

`git log --oneline --graph --all`	The whole history as a graph
`git log --grep="<text>"`	Find commits by message
`git diff main...<branch>`	What the branch changed since it split; the pull request view
`git blame -w <file>`	Who last touched each line, ignoring whitespace
`git bisect start`	Binary-search history for the commit that broke it
`git log -p -- <file>`	Every change to one file; --follow crosses renames
`git log -S "<text>"`	Find commits that added or removed this text
`git shortlog -sn --all`	Commits per author, ranked
`git show <hash>`	One commit in full; --stat for files only
`git branch -a --contains <hash>`	Which branches carry this commit

History surgery

Everything here rewrites history. Fresh clone first, backup kept, force-with-lease after, teammates re-clone.

`git commit --fixup <hash>`	Mark a fix as belonging to an earlier commit
`git rebase -i --autosquash <hash>^`	Melt fixups into their targets
`git filter-repo --invert-paths --path <file>`	Erase a file from all history
`git filter-repo --strip-blobs-bigger-than 10M`	Erase every giant blob
`git push --force-with-lease`	The only force push worth typing

Scale and speed

`git clone --filter=blob:none --sparse <url>`	Huge repository, download almost nothing
`git sparse-checkout set <folders>`	Materialize only what you work on
`git worktree add ../<repo>-review <branch>`	Second folder, second branch, zero stashing
`git lfs track "*.psd"`	Large binaries stored properly
`git count-objects -vH`	How heavy is this repository really
`git maintenance start`	Background upkeep keeps big repositories fast

Every platform, same Git

Git is identical everywhere; only the remote URL and sign-in differ. The same SSH public key works on every host.

`git clone git@github.com:<user>/<repo>.git`	GitHub over SSH
`git clone git@gitlab.com:<user>/<repo>.git`	GitLab
`git clone git@bitbucket.org:<workspace>/<repo>.git`	Bitbucket
`git clone https://dev.azure.com/<org>/<project>/_git/<repo>`	Azure DevOps (Azure Repos)
`git clone https://git-codecommit.<region>.amazonaws.com/v1/repos/<repo>`	AWS CodeCommit
`git config --global credential.helper manager`	Browser sign-in for HTTPS remotes, all hosts
`git remote set-url origin <url>`	Move between hosts or between HTTPS and SSH
`git clone --mirror <old> && git push --mirror <new>`	Migrate anywhere with full history

No host at all

Git predates the hosts and needs none of them.

`git init --bare ~/server/repo.git`	Your own remote on any machine or shared drive
`git bundle create repo.bundle --all`	The whole repository as one file: USB, email, air gap
`git clone repo.bundle -b main <folder>`	Clone from the file, no network
`git format-patch -3`	Last three commits as mailable patch files
`git am <file>.patch`	Apply mailed patches, authorship intact
`git archive --format=zip -o src.zip HEAD`	Clean source export, no .git

The errors you will meet

The message, then the cause. Every fix is one search away on the site.

`fatal: not a git repository (or any of the parent directories): .git`

This folder, and none above it, contains a .git directory. You are outside the project.

`git@github.com: Permission denied (publickey).`

The host never received a key it recognizes: no key loaded, key not added to your account, or an HTTPS problem disguised by an SSH URL.

`! [rejected] main -> main (non-fast-forward)`

The remote has commits you do not have. Git refuses to overwrite them.

`error: src refspec main does not match any`

The branch you are pushing does not exist locally, almost always because there is no commit yet.

`CONFLICT (content): Merge conflict in <file>`

Both sides changed the same lines; Git needs a human decision.

`remote: Repository not found.`

The URL has a typo, the repository is private and you lack access, or you are authenticated as the wrong account.

`fatal: Authentication failed for 'https://github.com/...'`

The credential your machine sent is wrong, expired, or a password where a token is required.

`! [rejected] main -> main (fetch first)`

Same cause as non-fast-forward: someone pushed before you.

`fatal: refusing to merge unrelated histories`

The two sides share no common commit, typically a fresh git init pulled against a remote that already has commits.

`error: Your local changes to the following files would be overwritten by merge`

The operation needs to rewrite files you have edited but not committed. Git refuses to destroy them silently.

Setup and identity

`git config --global user.name "<name>"`	Name stamped into your commits
`git config --global user.email "<email>"`	Use your host account email so commits link to your profile
`git config --global init.defaultBranch main`	New repositories start on main
`git config --global core.editor "code --wait"`	Commit messages open in VS Code
`git config --global pull.ff only`	Pull never creates surprise merge commits
`git config --global fetch.prune true`	Deleted remote branches disappear locally too
`git config --global rebase.autoStash true`	Pull with rebase works even with edits in progress
`git config --global core.autocrlf true`	Windows line endings handled once; macOS and Linux use input
`git config --list --show-origin`	Every setting and the file it comes from

Quality of life

`git config --global help.autocorrect prompt`	git statsu asks: run git status instead?
`git config --global alias.st "status -sb"`	git st forever
`git config --global alias.lg "log --oneline --graph --all"`	git lg: the graph
`git config --global rerere.enabled true`	Never resolve the same conflict twice
`git config --global gpg.format ssh`	First of three lines to signed, Verified commits
`git commit --allow-empty -m "rerun"`	Retrigger CI without touching code
`git config core.hooksPath .githooks`	Versioned hooks the whole team shares

The living version

Eight doors, one address: **amey-thakur.github.io/GIT-GUIDE**

Finder
ask anything, or paste the error

Start
zero to first push

Learn
the model, step by step

Fix
guided rescue

Errors
Git's messages, decoded

GitHub
first repo to branch protection

Workflows
how teams run Git

Cheatsheet
everything, one line each

Every answer also lives in the repository's Discussions; a question missing from the guide becomes its next addition. MIT licensed, no trackers, works offline.

Built and maintained by Amey Thakur
github.com/Amey-Thakur/GIT-GUIDE